

计算机图形学讲义笔记

高林

中国科学院计算技术研究所

2026.1.14

目录

1	课程简介	8
1.1	课程信息与教师介绍	8
1.2	前沿平台: Genesis	8
1.3	计算机图形学发展历史	9
1.4	AIGC 发展与几何表示演进	9
1.5	神经辐射场 (NeRF) 与高斯泼溅 (3D-GS)	10
1.6	几何表征的挑战与未来方向	10
1.7	计算机图形学定义与学科划分	10
1.8	图形软件系统	11
1.9	图形输出设备发展	12
1.9.1	CRT 显示器	12
1.9.2	光栅扫描显示器	12
1.9.3	GPU 发展历程	12
1.9.4	头戴式显示器	12
2	图形学基本知识	13
2.1	颜色视觉	13
2.1.1	颜色与光的谱分布	13
2.1.2	RGB 颜色空间	13
2.1.3	其他颜色空间	13
2.1.4	显示技术的色彩管理	13
2.2	图像与像素	13
2.3	三角网格模型	13
2.3.1	基本定义	14

2.3.2	法向量 (Normal Vector)	14
2.3.3	三角网格的绘制	14
2.4	光照模型与明暗处理	15
2.4.1	光照模型历史	15
2.4.2	Lambert 与 Half Lambert 模型	15
2.4.3	Phong 光照模型	15
2.4.4	明暗处理	16
2.5	变换	16
2.5.1	二维仿射变换的表示	16
2.5.2	齐次坐标改写	17
2.5.3	三维基本变换的齐次矩阵	18
2.5.4	法向量变换矩阵的推导	19
2.6	视点与投影	19
2.7	图形绘制管线	20
3	光栅图形学	21
3.1	光栅图形显示技术	21
3.2	走样与反走样	21
3.3	光栅化	21
3.4	简单二维图元的生成	21
3.5	直线的扫描转换算法	21
3.5.1	基本增量算法 (DDA 算法)	21
3.5.2	Bresenham 算法	22
3.6	圆的扫描转换	22
3.7	字符的生成	22
3.8	光栅图形的基本概念	23
3.9	区域填充	23
3.9.1	区域定义与连通性	23
3.9.2	种子填充算法	23
3.10	多边形的扫描转换	23
3.11	隐藏线/面消除算法	24
3.11.1	消隐分类	24
3.11.2	表优先级算法 (油画家算法)	25
3.11.3	Z 缓冲器算法	25
3.11.4	背面剔除算法	25

4	数字几何处理概论	26
4.1	几何处理流程概述	26
4.2	输入数据与坐标系统	26
4.3	三维模型数据结构	26
4.3.1	点云	26
4.3.2	体素	26
4.3.3	网格	26
4.3.4	常见数据格式	26
4.4	图像配准与 ICP 算法	27
4.4.1	基本步骤	27
4.4.2	每步优化的数学求解（基于 SVD 的封闭解）	27
4.4.3	3D 配准-先验	28
4.5	网格表示与存储	28
4.6	网格优化与平滑	28
4.6.1	孔洞填充	28
4.6.2	拉普拉斯平滑	28
5	数字几何处理一点云	30
5.1	点云的概念与特点	30
5.2	点云获取技术	30
5.3	点云处理流程	30
5.4	点一面 ICP 算法	30
5.5	点云法向估计与去噪	31
5.5.1	数学原理	31
5.5.2	PCA 法向估计的具体步骤	32
5.5.3	参数选择与影响	32
5.6	点云去噪	32
5.6.1	基本原理	32
5.6.2	算法流程	33
5.7	表面重建	33
5.7.1	隐式曲面方法	33
5.7.2	泊松表面重建	33
5.7.3	移动立方体算法	33
5.8	点云深度学习：PointNet	33
6	网格光顺、简化、细分与检索	34
6.1	网格光顺（去噪）	34
6.1.1	去噪的目的与难点	34

6.1.2	拉普拉斯光顺算法	34
6.2	网格简化	34
6.2.1	拓扑基本概念	34
6.2.2	简化方法分类	34
6.2.3	层次细节简化技术	35
6.2.4	基本化简操作	35
6.2.5	网格模型简化方法	35
6.3	网格细分	37
6.3.1	Loop 细分	38
6.3.2	$\sqrt{3}$ 细分	38
6.4	三维形状描述与检索	38
6.4.1	Spin Images 与 3D Shape Context	38
6.4.2	Shape Distributions 与相似性度量	38
7	网格变形与建模	39
7.1	网格变形概要	39
7.2	变形方法分类	39
7.3	基于 Laplacian 坐标的曲面变形	39
7.3.1	Laplacian 坐标定义	39
7.3.2	基本变形框架	39
7.3.3	旋转不变性问题	39
7.4	旋转传播方法	40
7.4.1	变形梯度与旋转提取	40
7.4.2	光滑旋转传播	40
7.5	空间变形：重心坐标法	40
7.5.1	基本思想	40
7.5.2	重心坐标的性质	40
7.5.3	常见重心坐标	40
7.6	ARAP 变形 (As-Rigid-As-Possible)	40
7.6.1	能量定义	40
7.6.2	迭代求解	41
7.7	骨骼蒙皮动画	41
7.8	应用与前沿	41
8	光线跟踪	43
8.1	光线跟踪的历史与发展	43
8.2	光线投射算法	43
8.3	光线跟踪算法原理	43

8.4	光线与几何体求交	44
8.4.1	光线表示	44
8.4.2	光线与平面求交	44
8.4.3	光线与三角形求交	44
8.4.4	光线与球面求交	45
8.4.5	光线与多边形求交	45
8.5	阴影计算	45
8.6	纹理映射	45
8.7	光线跟踪加速技术	45
8.7.1	包围盒技术	46
8.7.2	层次包围盒 (BVH)	46
8.7.3	包围盒求交算法	46
9	辐射度与渲染方程	47
9.1	预备知识	47
9.2	双向反射分布函数 (BRDF)	47
9.3	BRDF 模型	48
9.4	反射方程与渲染方程	48
9.4.1	反射方程	48
9.4.2	渲染方程	48
9.4.3	算子形式与级数解	48
9.5	蒙特卡洛路径追踪	49
9.5.1	蒙特卡洛积分	49
9.5.2	采样策略	49
9.5.3	路径追踪算法	49
9.5.4	拆分直接与间接光照	49
9.6	全局光照效果与实现	49
10	阴影与纹理	50
10.1	阴影	50
10.1.1	阴影的定义	50
10.1.2	阴影的类型	50
10.1.3	硬阴影与软阴影	50
10.1.4	平面阴影	50
10.1.5	曲面上的阴影	50
10.1.6	阴影绘制算法	51
10.2	纹理映射	51
10.2.1	纹理采集	51

10.2.2	纹理映射步骤	51
10.2.3	映射方式	51
10.2.4	纹理滤波与反走样	51
10.2.5	高级纹理技术	51
11	习题课: OpenGL 和光栅图形学实践	52
11.1	OpenGL 简介	52
11.1.1	发展历史与特点	52
11.1.2	OpenGL 的功能组成	52
11.2	OpenGL 体系结构与工作方式	52
11.2.1	体系结构层次	52
11.2.2	流水线与状态机制	52
11.2.3	OpenGL 组成与数据类型	52
11.3	GLUT 工具包	52
11.3.1	初始化与窗口创建	53
11.3.2	回调函数与事件循环	53
11.3.3	菜单创建	53
11.4	OpenGL 绘图基础	53
11.4.1	清除窗口	53
11.4.2	指定颜色与强制绘图	53
11.4.3	坐标系统与变换	53
11.5	光照处理	53
11.5.1	光照模型与材质属性	53
11.5.2	光照计算	54
11.6	几何图元绘制	54
11.6.1	顶点与图元定义	54
11.6.2	绕法	54
11.6.3	基本图元绘制示例	54
11.7	交互式绘图与菜单	54
11.8	三维模型表达与读取	54
11.8.1	模型格式 (OBJ)	54
11.8.2	模型读取与渲染	54
11.9	综合示例: 三维模型查看器	55
12	习题课: 光线追踪加速	56
12.1	光线追踪加速的必要性	56
12.2	光线追踪加速的基本思路	56
12.3	包围盒技术	56

12.3.1	轴向包围盒（AABB）与方向包围盒（OBB）	56
12.4	层次包围盒	56
12.4.1	BVH 的数据结构与构建	56
12.4.2	BVH 的相交测试	56
12.5	空间分割技术	56
12.5.1	均匀网格法	57
12.5.2	八叉树	57
12.5.3	KD 树	57
12.5.4	BSP 树	57
12.6	光线追踪实验代码分析	57
12.6.1	向量与几何类定义	57
12.6.2	光线追踪递归函数	57
12.6.3	相机投影与渲染循环	57
12.6.4	场景定义与结果	57
12.7	总结	58

1 课程简介

1.1 课程信息与教师介绍

主讲教师为高林，清华大学工学博士，导师胡事民院士。现为中国科学院计算技术研究所研究员、博士生导师。曾获国家优秀青年科学基金、北京市杰出青年科学基金、英国皇家学会牛顿高级学者等荣誉。研究方向包括计算机图形学、数字几何处理、三维计算机视觉等，在 ACM SIGGRAPH/TOG、IEEE TVCG、CVPR、TPAMI 等顶刊顶会发表多篇论文。

课程助教包括张凌霄（计算所工程师，研究方向为计算机图形学与几何重建）和周洁（计算所助理工程师，研究方向为几何分析与智慧医疗）。

考核方式

总成绩由开卷考试（40%）和平时考核（60%）构成。平时考核包括大作业、考勤与课堂表现。大作业涵盖基本图形技术、OpenGL 三维模型显示、网格简化、光线追踪，附加题为高斯泼溅重建立体模型。

课程定位

本课程为计算机科学与技术学院大三大四的专业选修课，总学时 38。教学目标是使学生掌握图形学基本原理与技术，能够开发三维应用，并具备初步的研究能力，了解如高斯泼溅等前沿技术。先修课程包括数据结构、离散数学、线性代数与程序设计。

参考书目

- 《计算机图形学基础教程》孙家广、胡事民，清华大学出版社，2009
- 《计算机图形学教程》唐荣锡等，科学出版社，2000
- 《Computer Graphics: Principles and Practice》Foley 等，Addison-Wesley，1997
- 《计算机图形学原理及实践》唐泽圣等译，机械工业出版社，2004
- 《3D 计算机图形学基础》夏时洪、高林译，2020

1.2 前沿平台：Genesis

Genesis 是一个面向机器人、嵌入式 AI 和物理 AI 的通用物理仿真平台，集成了通用物理引擎、机器人模拟、真实感渲染（内置光线追踪）以及自然语言驱动的多模态数据生成功能。

其愿景是降低物理仿真使用门槛，统一前沿物理求解器，并通过自动化数据生成降低数据收集成本。应用领域涵盖机器人策略训练、游戏开发、仿真教学、工业自动化、VR/AR 及影视动画等。

1.3 计算机图形学发展历史

计算机图形学发展经历了多个关键阶段：

- 1963 年：SketchPad 系统，首次实现光笔交互与矢量显示。奠定了人机交互和图形显示的基础，作者后来被称为“计算机图形学之父”。
- 1970 年代：纹理映射与 Z-Buffer 算法出现（Ed Catmull）。解决了“物体表面细节”和“前后遮挡关系”两大核心难题，是至今所有显卡光栅化渲染的基础。
- 1980 年：Turner Whitted 提出递归光线跟踪算法。
- 1986 年：Kajiya 提出渲染方程。
- 1995 年：Pixar 《玩具总动员》，影史第一部全电脑制作的长篇动画电影，标志着计算机图形技术在电影工业的成熟与大规模应用。
- 1999 年：NVIDIA 发布 GeForce 256，“GPU”（图形处理器）这一概念首次被提出。图形计算从 CPU 中解放出来，实时 3D 图形（如游戏）性能开始飞跃。
- 2020 年：NeRF（神经辐射场）提出。神经渲染的开山之作，打破了传统网格建模的思路，利用神经网络隐式表达场景，开启了 AI 与图形学结合的新赛道。
- 2023 年：3D Gaussian Splatting（3D 高斯泼溅）。解决了 NeRF 速度慢的问题，实现了实时、高质量的训练与渲染，是目前三维重建和渲染领域最前沿的技术爆发点（SIGGRAPH 2023 最佳论文）。

1.4 AIGC 发展与几何表示演进

AIGC 技术路线从变分自编码器（VAE）、生成对抗网络（GAN）、Transformer，到神经辐射场（NeRF）与扩散模型（Diffusion Model），再到近年出现的 SORA、高斯泼溅等系统。几何表示的关键要素可归纳为“REAL”：

- **R**: Photo-Realistic（照片级真实感）
- **E**: Efficient（高效）
- **A**: Scalable（可扩展）
- **L**: Low Cost（低成本）

几何表示的发展从样条曲线、点云、三角网格，到隐式场、神经辐射场（NeRF），再到高斯泼溅（3D-GS）及近年出现的 CLAY、高斯网等混合表示方式。

1.5 神经辐射场（NeRF）与高斯泼溅（3D-GS）

NeRF 简介

NeRF（神经辐射场）通过隐式表达和体渲染重建三维场景，输入为 (x, y, z, θ, ϕ) ，输出为 (RGB, σ) 。其优点是重建质量高、表达灵活，缺点为渲染速度慢、训练时间长。

团队在 NeRF 方向的研究涵盖人脸合成、编辑、重光照、动态重建等应用，并在加速与质量提升方面提出了 Tri-MipNeRF、DE-NeRF 等方法。

3D-GS 简介

3D 高斯泼溅通过稀疏三维点云与光栅化实现实时渲染，渲染帧率可达 135fps，训练速度快（约 6 分钟），兼容传统渲染管线。相比 NeRF，3D-GS 在渲染效率上优势明显，但 NeRF 在复杂材质重建与解耦方面仍有优势。

团队在 3D-GS 方向的研究涵盖双目 VR、数字人、自动驾驶等应用，并开展了材质、光照、风格编辑以及几何表示与渲染增强等工作。

1.6 几何表征的挑战与未来方向

高斯泼溅面临的几何挑战包括离散性与拓扑缺失、复杂材质重建困难以及稀疏输入下的可控生成问题。

未来趋势倾向于混合表示（如 3D-GS 与隐式场结合）与生成式重建（如 CAST、VEGS）。理想汽车等企业已在自动驾驶中采用“重建 + 生成”的世界模型。

1.7 计算机图形学定义与学科划分

计算机图形学是研究图形的表示、生成、处理与显示的学科。主要分为四个子领域：

- 图像（Imaging）：表示二维图像，包含滤波、变形、合成等技术
- 建模（Modeling）：表示三维物体（描述形状和表面参数），包含曲面、平面、实心模型等
- 渲染（Rendering）：从三维物体中生成二维图像，包含相机模型、隐藏表面消除、光线传播等技术
- 动画（Animation）：模拟时间上的变化，包含运动学、动力学、规划、学习等

与相关学科的区别在于：

- 计算机图形学 (**CG**): 输入描述, 输出图像), 目的在于生成图像。
- 计算机视觉 (**CV**): 输入图像, 输出描述, 目的在于理解图像, 获得语义信息。(“以图搜图”)
- 数字图像处理 (**DIP**): 输入图像, 输出图像, 目的在于转换图像。
- 模式识别 (**PR**): 从特征空间映射到类别空间, 研究内容包括特征提取 (PCA, LDA, LFA, Kernel, Mean Shift, SIFT, ISOMAP, LLE)、特征选择、分类器设计 (SVM, AdaBoost) 等。
- 机器视觉 (**MV**): 工业领域的视觉研究。通过软硬件, 图像感知与控制理论往往与图像处理得到紧密结合来实现高效的机器人控制或各种实时操作。

1.8 图形软件系统

图形软件系统应具有良好的层次结构和模块结构, 以便于设计、维护和调试。系统通常分为以下四个层次, 其中 0-2 级统称为“支撑软件”:

零级图形软件 (底层/驱动层)

是最底层的软件, 面向系统。主要功能是解决图形设备与主机的通讯、接口问题, 包含基本的输入/输出子程序, 也称为设备驱动程序。

一级图形软件 (基础层)

既面向系统又面向用户。功能包括生成基本图形元素 (图素) 和设备管理, 也称为基本子程序。

二级图形软件 (功能层)

面向用户, 具有较强的交互功能。在基础层之上构建, 负责建立图形数据结构, 定义、修改和输出图形, 建立设备间的联系, 也称为功能子程序。

三级图形软件 (应用层)

是整个应用软件的一部分, 通常由用户或与设计者共同编写。主要功能是解决具体的应用问题。

基本图形软件的内容与构建

基本图形软件作为图形系统的核心支撑，一般应包含以下模块：

1. 系统管理程序：负责整体协调与管理
2. 图素生成程序：定义和输出基本图素及复合图素
3. 图形变换程序：包括几何变换、开窗（Windowing）、裁剪（Clipping）等
4. 实时输入处理程序：处理来自设备的实时数据
5. 交互处理程序：支持用户与系统的交互操作

构建基本图形软件通常有三种主要途径：

1. 图形程序包：在现有的高级语言基础上增加图形库/程序包（最常见）
2. 修改高级语言：对现有语言进行扩展以支持图形功能
3. 专用图形语言：开发专门用于图形处理的高级语言

1.9 图形输出设备发展

1.9.1 CRT 显示器

单色 CRT 显示图像的质量取决于：设备固有的单个光点直径的大小以及“可寻址能力”。可寻址能力可以理解为单位长度内能够利用的单个光点的数目。通常希望点的直径大于点间距。一个 CRT 在水平或垂直方向上能够识别出的最大光点数称为分辨率。

彩色 CRT 中，射线穿透法用于随机扫描显示器，影孔板法用于光栅扫描系统。

1.9.2 光栅扫描显示器

屏幕分为 m 行扫描线，每行 n 个像素，通过帧缓存存储像素亮度或色彩信息。其优点是可显示面图形且成本低，缺点是转换耗时、软件复杂。

1.9.3 GPU 发展历程

从 1999 年 NVIDIA 提出 GPU 概念，到可编程处理器出现、CUDA 发布，再到实时光线追踪架构与 ARM 处理器发布。

1.9.4 头戴式显示器

从早期 Sensorama 到第一款 AR/VR 头盔，再到 Apple Vision Pro 混合现实设备。

2 图形学基本知识

2.1 颜色视觉

2.1.1 颜色与光的谱分布

颜色是人眼对不同波长光的感知，可见光波长范围为 380nm 到 760nm。光的谱分布描述各波长分量的强度分布。异谱同色现象指不同谱分布可能对应相同颜色感知。

2.1.2 RGB 颜色空间

RGB 颜色空间采用红、绿、蓝三基色，基于人类视觉的三刺激理论。颜色用三通道向量 (r, g, b) 表示，规整化为 $[0, 1]$ 或 $[0, 255]$ 。其缺点是部分色彩无法表示为 RGB 的正线性组合。

2.1.3 其他颜色空间

常用颜色空间还包括：

- **CMY**：青、品红、黄，属减色系统。
- **HSV**：色调（描述色彩的本征属性）、饱和度（饱和度越低，色彩越白）、亮度（亮度越低，色彩越黑），更用户友好。
- **CIE XYZ**：可表示所有可感知颜色，适用于颜色科学研究。

2.1.4 显示技术的色彩管理

LCD 依赖背光与液晶分子，通过控制光线透过率实现显示。LED 为其升级版，采用微型 LED 灯珠作为发光源，通过缩小灯珠间距提升分辨率。OLED 则通过自发光像素实现更高对比度，量子点增强 (QD-OLED) 和烧屏防护技术降低了使用风险。

2.2 图像与像素

图像可视为二维离散函数 $f(x, y)$ ，定义域为像素矩阵，取值为 RGB/RGBA 或灰度值。

2.3 三角网格模型

三角网格是计算机图形学中最常用的三维模型表示方法，由一系列三维顶点及连接这些顶点的三角面片组成。简单几何体可直接用参数描述，而复杂模型通常需要使用三角网格离散化表示，以便于渲染和处理。

2.3.1 基本定义

形式化定义为：

- 顶点集合： $V = \{v_1, v_2, \dots, v_n\}$ ，其中每个 $v_i \in \mathbb{R}^3$ 表示空间中的一个点
- 面片集合： $F = \{f_1, f_2, \dots, f_m\}$ ，每个 f_i 是由 V 中的顶点构成的空间三角形
- 例如： $f_1 = (v_{a1}, v_{b1}, v_{c1}), f_2 = (v_{a2}, v_{b2}, v_{c2}), \dots$

2.3.2 法向量 (Normal Vector)

法向量是进行光照处理的必要输入，决定了模型的三维立体感。

三角面片的法向量

定义为垂直于该三角面片所在平面的非零向量。每个面片有两个可能的法向朝向，决定了面片的正面与反面。对于连续可定向的网格，相邻面片的法向朝向应保持一致。

顶点法向量的计算

为使渲染表面光滑，顶点法向量通常由其周围邻接的三角面片的法向量加权计算。假设顶点 v 是 k 个三角面片 $f_1, f_2, \dots, f_k$ 的公共顶点：

1. 算术平均法：简单地对周围面片的法向量求平均：

$$N_v = \frac{N_{f1} + N_{f2} + \dots + N_{fk}}{k}$$

2. 面积加权平均法：根据面片面积 S_{fi} 加权，面积越大的面片贡献越大：

$$N_v = \frac{S_{f1}N_{f1} + S_{f2}N_{f2} + \dots + S_{fk}N_{fk}}{S_{f1} + S_{f2} + \dots + S_{fk}}$$

3. 角度加权平均法：根据面片在顶点处所占的角度 Ang_{fi} 加权：

$$N_v = \frac{Ang_{f1}N_{f1} + Ang_{f2}N_{f2} + \dots + Ang_{fk}N_{fk}}{Ang_{f1} + Ang_{f2} + \dots + Ang_{fk}}$$

2.3.3 三角网格的绘制

三角网格主要有三种绘制方式：

线框模式 (Wireframe)：仅绘制三角形的边，显示模型的基本结构。

基于颜色的绘制：为每个顶点指定颜色属性，三角形内部像素的颜色通过顶点颜色插值得到。

基于光照的绘制：指定虚拟光照环境，结合法向量和光照模型计算颜色，体现立体感。

2.4 光照模型与明暗处理

光照模型用于计算光的强度。局部光照明关注物体直接受到光源影响所产生的光照效果。全局光照明关注阴影效果，以及所有不是直接与光源位置相关的光照效果，例如反射和折射效果，等等。

2.4.1 光照模型历史

从 Wylie 的距离反比光照、Bouknight 的 Lambert 漫反射加环境光、Gouraud 的插值思想，到 Phong 光照模型，逐步完善了真实感渲染。

2.4.2 Lambert 与 Half Lambert 模型

Lambert 模型模拟粗糙表面漫反射：

$$I_d = K_d \cdot I \cdot \text{dot}(N, L)$$

式中 K_d 是漫反射系数，与模型自身的色彩紧密相关，其具有三个分量 k_{dr}, k_{dg}, k_{db} 分别代表 R, G, B 三个通道的漫反射系数。 N 为表面法向量， L 为光源方向向量，二者均为单位向量，其点积为光线与法线的夹角。 I 为入射光强度， I_d 为漫反射光强度。

Half Lambert 通过线性映射提升整体亮度：

$$I_d = K_d \cdot I \cdot (\text{dot}(N, L) \cdot 0.5 + 0.5)$$

2.4.3 Phong 光照模型

Phong 模型将光照分解为环境光、漫反射光与镜面反射光三部分：

$$I = I_{amb}K_a + I_iK_s(R \cdot V)^n + I_iK_d(L \cdot N)$$

$I_{amb}K_a$ 表示环境光，即周围环境的散射光提供的间接光照，其为一个常量，用于照亮阴影区域，防止背光面变成纯黑。其中 K_a 为环境反射系数，决定物体对环境光的吸收率。 I_{amb} 为环境光强度。

$I_iK_d(L \cdot N)$ 表示基于 Lambert 模型的漫反射光。

$I_iK_s(R \cdot V)^n$ 表示镜面反射光，即高光。 I_i 为入射光强度， K_s 为镜面反射系数，决定物体表面对镜面反射光的吸收率。 R 为反射向量， V 为视线方向向量，二者均为单位向量，其点积为视线与反射光的夹角。 n 为高光指数，决定高光的集中程度，值越大高光越集中，说明物体表面非常光滑、抛光。

2.4.4 明暗处理

为消除离散化带来的不光滑感,采用 Gouraud 明暗处理(先计算每个顶点光强,再利用双线性插值¹计算多边形表面上每个像素明暗)或 Phong 明暗处理(先双线性插值计算每个像素点法向量,然后用其计算像素光强),后者能产生更精确的高光效果。

2.5 变换

变换是空间点之间的映射,分为刚体变换(保持度量,如不变、平移、旋转及其复合)、相似变换(保持角度,包括刚体变换与均衡缩放)、线性变换(满足 $L(p+q) = L(p) + L(q)$ 且 $L(\alpha p) = \alpha L(p)$ 的映射,包含不变、旋转、缩放、对称、错切)、仿射变换(保持直线以及直线与直线的平行,包含线性变换与相似变换)与投影变换(保持直线,包含前述所有)等类型。

2.5.1 二维仿射变换的表示

一般形式

任何保持“平直性”(直线变换后仍为直线)和“平行性”的二维几何变换,称为仿射变换。其最一般的形式由两个线性方程组合一个平移项构成:

$$x' = ax + by + c \quad (4)$$

$$y' = dx + ey + f \quad (5)$$

其中:

- 系数 a, b, d, e 构成的线性部分 $\begin{bmatrix} a & b \\ d & e \end{bmatrix}$ 控制旋转、缩放、错切、反射等。
- 常数项 c, f 构成平移向量 $\begin{bmatrix} c \\ f \end{bmatrix}$, 实现位置的移动。

这种形式直接体现了仿射变换是“线性变换 + 平移”的复合。

¹双线性插值公式如下,设已知点 $Q_{11}(x_1, y_1), Q_{21}(x_2, y_1), Q_{12}(x_1, y_2), Q_{22}(x_2, y_2)$ 的函数值 $f(Q_{ij})$, 需计算点 $P(x, y)$ 的函数值 $f(P)$, 其中 $x_1 \leq x \leq x_2, y_1 \leq y \leq y_2$:

$$\text{第一步: 水平插值} \quad f(R_1) = \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21}) \quad (1)$$

$$f(R_2) = \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22}) \quad (2)$$

$$\text{第二步: 垂直插值} \quad f(P) = \frac{y_2 - y}{y_2 - y_1} f(R_1) + \frac{y - y_1}{y_2 - y_1} f(R_2) \quad (3)$$

即可得到点 P 的函数值 $f(P)$ 。

矩阵形式的统一

将 (4) 和 (5) 改写为矩阵乘法形式：

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ d & e \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} c \\ f \end{bmatrix}$$

记为更简洁的形式：

$$\mathbf{p}' = \mathbf{M}\mathbf{p} + \mathbf{t} \quad (6)$$

2.5.2 齐次坐标改写

定义与动机

直接使用 (6) 形式无法通过单一的矩阵乘法统一表示平移。为此引入齐次坐标：将 n 维点提升到 $n+1$ 维，并在末尾添加一个分量 w （通常设为 1）。

$$2\text{D 点}(x, y) \rightarrow \text{齐次坐标}(x, y, 1)$$

在齐次坐标下，仿射变换可统一表示为单一的 3×3 矩阵乘法：

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

此矩阵称为仿射变换的齐次矩阵，其结构为：

$$\mathbf{A} = \begin{bmatrix} \mathbf{M} & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}$$

齐次坐标中 w 维度的性质

1. 常态值：在大部分情况下（表示有限远点），我们设置 $w = 1$ 。此时前 n 个坐标即普通坐标， w 可被“忽略”（但作为占位符存在）。
2. 变换下的行为：
 - 当齐次坐标被 仿射矩阵（形如上式，末行为 $[0 \ 0 \ 1]$ ）作用时，由于最后一行固定为 $[0 \ 0 \ 1]$ ， w 分量保持不变（ $w' = 0 \cdot x + 0 \cdot y + 1 \cdot w = w$ ）。
 - 当被 投影矩阵（末行不全为 $[0 \ 0 \ 1]$ ，例如透视投影矩阵）作用时， w 分量会发生改变。
3. 归一化与等价性：

- 当 $w \neq 0$ 时，齐次坐标 (x, y, z, w) 表示的是同一个点的不同表示。通过同时除以 w 进行归一化：

$$(x, y, z, w) \sim \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w}, 1 \right)$$

例如， $(2x, 2y, 2z, 2) \sim (x, y, z, 1)$ 。

- 当 $w = 0$ 时，齐次坐标 $(x, y, z, 0)$ 不再表示一个有限远点，而是表示一个沿向量 (x, y, z) 方向的无穷远点（或视为一个方向向量）。它无法被归一化为 $w = 1$ 的形式。

2.5.3 三维基本变换的齐次矩阵

在三维齐次坐标 $(x, y, z, 1)$ 下，基本变换的 4×4 齐次矩阵如下：

- 平移（平移量 t_x, t_y, t_z ）：

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- 缩放（缩放因子 s_x, s_y, s_z ）：

$$\mathbf{S} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- 绕坐标轴的旋转（ θ 为旋转角度，符合右手定则）：

$$\text{绕 } x \text{ 轴: } \mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{绕 } y \text{ 轴: } \mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\text{绕 } z \text{ 轴: } \mathbf{R}_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.5.4 法向量变换矩阵的推导

对于表面上的点，其变换规则为 $\mathbf{p}' = \mathbf{M}\mathbf{p}$ （在齐次坐标下）。然而，法向量 $\mathbf{n}$ 的变换矩阵并非 $\mathbf{M}$ ，而是 $\mathbf{M}$ 的逆的转置： $\mathbf{n}_{\text{WS}} = (\mathbf{M}^{-1})^T \mathbf{n}_{\text{OS}}$ 。

推导过程

原始垂直条件： $\mathbf{n}^T \mathbf{t} = 0$ 。

变换后的切向量： $\mathbf{t}' = \mathbf{M}\mathbf{t}$ 。

设变换后的法向量为 $\mathbf{n}' = \mathbf{A}\mathbf{n}$ ，其中 $\mathbf{A}$ 是待求的变换矩阵。

新的垂直条件： $(\mathbf{n}')^T \mathbf{t}' = 0$ ，代入得：

$$(\mathbf{A}\mathbf{n})^T (\mathbf{M}\mathbf{t}) = \mathbf{n}^T \mathbf{A}^T \mathbf{M} \mathbf{t} = 0$$

由于 $\mathbf{n}^T \mathbf{t} = 0$ 对于所有切向量 $\mathbf{t}$ 都成立，要使新条件恒成立，需有：

$$\mathbf{A}^T \mathbf{M} = k\mathbf{I} \quad (k \text{ 为标量，通常取 } 1 \text{ 以保持长度或方向不变})$$

由此可得：

$$\mathbf{A}^T = \mathbf{M}^{-1} \Rightarrow \mathbf{A} = (\mathbf{M}^{-1})^T$$

因此，当顶点位置由矩阵 $\mathbf{M}$ 变换时，其关联的法向量必须用矩阵 $(\mathbf{M}^{-1})^T$ 进行变换，才能保证变换后法向量与曲面仍然垂直：

$$\mathbf{n}_{\text{WS}} = (\mathbf{M}^{-1})^T \mathbf{n}_{\text{OS}}$$

其中下标 OS 表示物体局部空间，WS 表示世界空间。

2.6 视点与投影

投影分为正交投影与透视投影。当 xy 平面是投影平面时，正交投影把 (x, y, z) 映射成为 $(x, y, 0)$ 。当视点位于原点，投影平面为 $z = d$ 时，透视投影矩阵将 (x, y, z) 映射为 $((d/z)x, (d/z)y, d)$ 。

视图变换 (View Transform)

视图变换的目的是确定相机的位置和朝向。给定眼睛位置 eye 、参考位置 $center$ 和向上方向 up 。

构建相机坐标系的基底向量 z^c, x^c, y^c ：

$$\mathbf{z}^c = \frac{eye - center}{\|eye - center\|} \quad (\text{视线反方向}) \quad (7)$$

$$\mathbf{x}^c = \frac{up \times \mathbf{z}^c}{\|up \times \mathbf{z}^c\|} \quad (\text{右方向}) \quad (8)$$

$$\mathbf{y}^c = \mathbf{z}^c \times \mathbf{x}^c \quad (\text{重新计算的上方向}) \quad (9)$$

视图变换矩阵 M 由平移矩阵 $T(-e)$ 和旋转矩阵 R 复合而成（先平移到原点，再旋转）：

$$M = R \cdot T(-e) \quad (10)$$

其中 $T(-e)$ 将相机平移至世界坐标系原点， R 将世界坐标系的轴旋转对齐到相机坐标系的轴 (x^c, y^c, z^c) 。

矩阵展开形式如下：

$$M = \begin{bmatrix} x_x^c & x_y^c & x_z^c & 0 \\ y_x^c & y_y^c & y_z^c & 0 \\ z_x^c & z_y^c & z_z^c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & -eye_x \\ 0 & 1 & 0 & -eye_y \\ 0 & 0 & 1 & -eye_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (11)$$

合并后的矩阵形式：

$$M = \begin{bmatrix} x_x^c & x_y^c & x_z^c & -(x_x^c eye_x + x_y^c eye_y + x_z^c eye_z) \\ y_x^c & y_y^c & y_z^c & -(y_x^c eye_x + y_y^c eye_y + y_z^c eye_z) \\ z_x^c & z_y^c & z_z^c & -(z_x^c eye_x + z_y^c eye_y + z_z^c eye_z) \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (12)$$

2.7 图形绘制管线

绘制管线包括顶点着色器、物体变换、视图变换、投影变换、标准化设备坐标（NDC）与视口变换等阶段。

透视投影矩阵（对称视锥）为：

$$M_{proj} = \begin{bmatrix} \frac{f}{aspect} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & \frac{zFar+zNear}{zFar-zNear} & \frac{2 \cdot zFar \cdot zNear}{zFar-zNear} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

其中 $f = \cot(fovy/2)$ 。

正交投影矩阵为：

$$M_{ortho} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{n-f} & -\frac{n+f}{n-f} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

视口变换将 NDC 映射到屏幕像素坐标。

3 光栅图形学

3.1 光栅图形显示技术

光栅显示器通过一组平行的水平扫描线显示图像，每条扫描线由大小一致的像素构成。图像以像素形式存储在刷新缓冲器中。

像素与图元

像素是最小可编程物理单元，通常为四边形或六角形。图元是描述图形元素的基本函数（如点、直线），而圆等曲线通常不是基本图元。

随机扫描与光栅扫描对比

随机扫描按显示命令顺序偏转电子束，屏幕上的点不可编程；光栅扫描中像素可编程，支持并行处理与区域填充。光栅扫描因可编程性与并行能力成为主流。

3.2 走样与反走样

走样现象表现为用像素近似平滑图元时出现的“锯齿状”边界。反走样的核心思想是“先过滤后采样”（如在采样前模糊），通过引入中间颜色值缓解视觉突兀感。

超采样过程把当前分辨率成倍提高，然后再把画缩放到当前的显示器上，从而逼近得到更好的效果，实现反走样。自适应超采样只在当像素在物体边沿时进行超级采样，从而降低开销。

3.3 光栅化

光栅化是确定最佳逼近图形的像素集合，并用指定的颜色和灰度设置像素的过程，也称为“扫描转换”，目标是使光栅图形最佳逼近实际图形。

3.4 简单二维图元的生成

OpenGL 绘制过程包括：定义几何要素、确定物体位置与观察点、计算表面颜色、光栅化四个步骤。

3.5 直线的扫描转换算法

3.5.1 基本增量算法（DDA 算法）

基于微分方程 $dy/dx = m$ ，沿 x 方向步进， y 增加斜率 m ，对于每个 x 画出最近的整数 y 。对大斜率直线需交换 x 与 y 角色以保证连续性。

3.5.2 Bresenham 算法

通过各行、各列像素中心构造一组虚拟网络线。按照直线起点到终点的顺序，计算直线与各垂直网络线的交点，然后根据误差项的符号确定该列像素中于此交点最近的像素。仅使用整数运算，通过误差项判断 y 是否递增。

初始版本误差项 d 的初值为 0， $d = d + k$ (k 为斜率)，一旦 $d \geq 1$ 则 y 递增且 $d = d - 1$ ，保证 d 的相对性且在 0、1 之间。若 $d > 0.5$ ，说明交点在像素上方，选择上方像素；否则选择下方像素。

改进版本 1 令 $e = d - 0.5$ ，初值 $e = -0.5$ ，每次更新 $e = e + k$ ，若 $e > 0.5$ 则 y 递增且 $e = e - 1$ 。 $e > 0$ 时， y 方向递增 1； $e < 0$ 时， y 方向不递增。 $e = 0$ 时，可任取上下光栅点显示。

改进版本 2 用 $2dx \cdot e$ 替换误差项，完全避免浮点运算：

1. 输入端点 $P_0(x_0, y_0), P_1(x_1, y_1)$
2. 初始化 $dx, dy, e = -dx, x = x_0, y = y_0$
3. 绘制点 (x, y)
4. 更新 $e = e + 2dy$ ，若 $e > dx$ 则 $(x, y) = (x + 1, y + 1), e = e - 2dx$ ；否则 $(x, y) = (x + 1, y)$
5. 重复直到直线绘制完毕

3.6 圆的扫描转换

利用八分对称性，只需绘制第一象限 $y = x$ 上方的 1/8 圆弧。

对于当前点 (x_i, y_i) ，下一像素为 $(x_i + 1, y_i)$ 或 $(x_i + 1, y_i - 1)$ 。取中点 $(x_i + 1, y_i - 0.5)$ 判断：

$$d_i = (x_i + 1)^2 + (y_i - 0.5)^2 - r^2$$

若 $d_i < 0$ ，取 $y_{i+1} = y_i$ ；否则取 $y_{i+1} = y_i - 1$ 。

递推公式：

- 若 $d_i < 0$ ： $d_{i+1} = d_i + 2x_i + 3$
- 若 $d_i \geq 0$ ： $d_{i+1} = d_i + 2(x_i - y_i) + 5$

初始条件： $x_0 = 0, y_0 = r, d_0 = 1.25 - r$ 。

3.7 字符的生成

字符生成主要有轮廓法（定义为曲线或多边形轮廓，开销大）和位图法（生成矩形位图，1 表示笔画经过）。

3.8 光栅图形的基本概念

光栅图形的本质是点阵表示。其采用面着色，画面明暗自然、色彩丰富，与线框图相比更加生动、直观、真实感强。

图形学中的多边形指无自相交的简单多边形。其有两种表示方式：顶点表示（有序顶点序列，几何意义明确、开销小但不便着色）和点阵表示（内部像素集合，便于着色与帧缓冲器表示但丢失几何信息、开销大）。

3.9 区域填充

3.9.1 区域定义与连通性

区域：已经表示成点阵的像素集合。可分为内部表示（枚举所有像素）和边界表示（枚举边界像素）。连通性包括四连通（上下左右）和八连通（包含对角线方向）。

3.9.2 种子填充算法

递归填充四连通区域，算法检查当前像素颜色，若为待填充色则着色并递归调用相邻像素。在边界表示区域时，需先将边界像素着色，递归调用时除检查是否不为填充色外，还需检查边界色。

3.10 多边形的扫描转换

多边形的扫描转换就是把顶点表示转换为点阵表示。从多边形的给定边界出发，求出其内部的各个像素，并给帧缓冲器中各个对应元素设置相应灰度或颜色。

逐点判断算法

逐个像素判断是否在多边形内部，使用射线法计算交点个数（奇内偶外）。效率较低，且未利用像素连贯性。

扫描线算法

区域连贯性：多边形定义的区域内部相邻的像素具有相同的性质。两条扫描线之间的长方形区域被所处理的多边形分割成若干梯形，相邻梯形必有一个位于多边形内部，另一个在多边形外部。如果上述梯形属于多边形内（外），那么该梯形内所有点的均属于多边形内（外），从而实现由逐点判断转化为区域判断。

扫描线连贯性：区域连贯性在一条扫描线上的反映。扫描线截多边形所得的线段与截完剩下的线段相间排列，如果扫描线与多边形的某两个交点确定的线段属于多边形内（外），那么该区间内所有点均属于多边形内（外），从而实现由逐点判断转化为区间判断。

边的连贯性：直线的线性性质在光栅上的表现。当知道扫描线与多边形一个边的交点后，利用这条边的斜率可以通过增量算法迅速求出其他交点。

扫描线连贯性 “+” 边连贯性 “=” 区域连贯性。

扫描线算法填充多边形的基本思想是按扫描线顺序，计算扫描线与多边形的相交区间，再用要求的颜色显示这些区间的像素，即完成填充工作。

1. 确定多边形占据的扫描线范围 $[y_{\min}, y_{\max}]$
2. 从 $y_{\min}$ 到 $y_{\max}$ （即从下往上）每次用一条扫描线进行填充：先求与多边形各边的交点，再按 x 递增排序后配对，对配对区间内像素着色，区间外着背景色。

当扫描线与多边形顶点相交时，根据共享边的相对位置决定交点计数。

若共享顶点的两条边分别落在扫描线的两边，交点只能算一个；若共享顶点的两条边在扫描线的同一边，这时交点作为 0 个或者 2 个。

检查共享顶点的两条边的另外两个端点的 y 值：若两端点 y 值小于交点 y 值（上凸点），交点作为 0 个；两端点 y 值大于交点 y 值（下凸点），交点作为 2 个。

3.11 隐藏线/面消除算法

消隐：相对于观察者，确定场景中哪些物体是可见或部分可见的，哪些物体是不可见的。

消隐与物体排序（判断场景中的物体全部或者部分与视点之间的远近）、连贯性（场景中物体或其投影所表现出来的相似程度）密切相关。消隐算法的效率很大程度上取决于排序的效率和各种连贯性的利用。在实时模拟过程中，要求消隐算法速度快，通常生成的图形质量一般；在真实感图形生成过程中，要生成高质量的图形，通常消隐算法速度较慢。

3.11.1 消隐分类

按消隐对象分为线消隐（输出线框图）和面消隐（输出着色图）。

按坐标空间分为：

- 图像空间消隐：屏幕坐标系中进行，生成的图像一般受限于显示器的分辨率。复杂度 $O(nN)$ （ n 为物体数， N 为像素数），场景中每一个物体要和屏幕中每一个像素进行排序比较。代表如 Z 缓冲器算法，扫描线算法。
- 物体空间消隐：世界坐标系中进行，算法精度高，与显示器的分辨率无关，适合于精密的 CAD 工程领域。复杂度 $O(n^2)$ （ n 为物体数），场景中每一个物体都要和场景中其他的物体进行排序比较。代表如背面剔除、表优先级算法等。

理论上如果 $n < N$ ，则景物空间算法的计算量 $O(n^2)$ 小于图像空间算法 $O(nN)$ 。但在实际应用中通常会考虑画面的连贯性，所以图像空间算法的效率有可能更高。

3.11.2 表优先级算法（油画家算法）

在物体空间确定物体之间的可见性顺序 (物体离视点远近), 由远及近地绘制出正确的图像结果, 使得离视点近的物体可能遮挡离视点远的物体。要求场景中物体在 z 方向上没有相互重叠。二维半物体 (卡通动画、窗口管理等) 的深度值是常数, 排序算法只要简单地比较其 z 值即可。

3.11.3 Z 缓冲器算法

为每个像素维护深度值, z 值越大离视点越近。绘制时比较当前像素深度与缓冲区存储值, 更新更近的像素。优点是适合于任何几何物体、无需排序、适合并行; 缺点是存储占用大、难以处理透明物体。逐区域进行 z 缓冲器消隐可以优化存储。

算法步骤:

1. 初始化帧缓冲器中的颜色为背景色, z 缓冲器中的 z 值为最小值 (离视点最远)
2. 以任意顺序扫描多边形, 对多边形每个像素计算深度 $z(x, y)$ 。若 $z(x, y) > z_{\text{buffer}}(x, y)$, 更新颜色和深度缓冲区。

当将视点放置光源时, Z 缓冲器算法可用于阴影生成。

3.11.4 背面剔除算法

适用于凸多面体, 利用法向量与视线方向点积判断: $N \cdot V < 0$ 表示不可见面。场景消隐中可用于预处理, 消除一些显然不可见表面, 从而提高其它消隐算法的效率。

4 数字几何处理概论

4.1 几何处理流程概述

几何处理是从输入数据到最终模型输出的一系列计算与操作，包括数据获取、表面重建、表面质量分析、表面平滑与去噪、参数化、简化以降低复杂度、重网格化以提高网格质量、自由变形与多分辨率建模等步骤。

4.2 输入数据与坐标系统

输入数据来源包括实体模型三维扫描、CAD 人机交互、CT/MRI 医学影像和计算机模拟生成。

常用坐标系有直角坐标系（分左手系与右手系）、球坐标系 (r, θ, ϕ) 和柱坐标系 (r, h, θ) 。局部坐标系表示简洁、便于重复使用与几何操作，世界坐标系与局部坐标系间可通过线性变换转换。

4.3 三维模型数据结构

4.3.1 点云

存储每个点的 (x, y, z) 坐标，可包含颜色、时间等信息，常用于三维激光扫描、影像重建与逆向工程。获取方式包括直接采集（激光扫描）和从多视角二维图像重建。

4.3.2 体素

是数字数据于三维空间分割上的最小单位，类似于二维图像中的像素。

4.3.3 网格

多边形网格是三维物体的常见表示，三角形网格最为普遍。网格由顶点和面片构成，要求相邻面片共享完整的边。

4.3.4 常见数据格式

- **OBJ** 格式：支持顶点、纹理、法向、面片与材质信息，支持曲线曲面。
- **STL** 格式：用于 3D 打印，仅描述几何信息。
- **PLY** 格式：结构简单，常用于 3D 扫描数据。

4.4 图像配准与 ICP 算法

配准问题，即给定两个部分重叠的点云或网格，找到它们之间的最优空间对齐（刚性变换）。图像配准常用迭代最近点（ICP）算法。

4.4.1 基本步骤

1. 在源点云中采样。
2. 对源点云中的每个点，在目标点云中找到离它最近的点 $y_i = \arg \min_{y \in Y} \|y - x_i\|$ 。
3. 构建目标函数 $E = \sum \|Rx_i + t - y_i\|^2$ ，以最小化之为目标优化旋转矩阵 R 与平移向量 t ，迭代至收敛。

4.4.2 每步优化的数学求解（基于 SVD 的封闭解）

给定两组配对好的三维点云集合：

- 源点云 (Source): $X = \{x_1, x_2, \dots, x_N\}$
- 目标点云 (Target): $Y = \{y_1, y_2, \dots, y_N\}$

其中 x_i 与 y_i 是对应点。我们的目标是寻找一个旋转矩阵 R 和平移向量 t ，使得下述最小二乘误差函数最小化：

$$E(R, t) = \sum_{i=1}^N \|Rx_i + t - y_i\|^2 \quad (13)$$

首先计算两个点云的质心（均值）：

$$\mu^X = \frac{1}{N} \sum_{i=1}^N x_i, \quad \mu^Y = \frac{1}{N} \sum_{i=1}^N y_i \quad (14)$$

构建协方差矩阵 C （注意此处令顺序为 目标 $\times$ 源的转置）：

$$C = \sum_{i=1}^N (y_i - \mu^Y)(x_i - \mu^X)^T \quad (15)$$

对矩阵 C 进行奇异值分解（SVD）：

$$C = U\Sigma V^T \quad (16)$$

根据 SVD 的结果，最优旋转矩阵为：

$$R_{opt} = U\tilde{\Sigma}V^T \quad (17)$$

其中 $\tilde{\Sigma}$ 用于处理反射（Reflection）情况，确保行列式为 1：

- 如果 $\det(UV^T) = 1$, 则 $\tilde{\Sigma} = I$ (即 $R_{opt} = UV^T$)。
- 否则 ($\det(UV^T) = -1$), $\tilde{\Sigma} = \text{diag}(1, 1, \dots, -1)$ 。

一旦求得最优旋转矩阵, 平移向量由质心对齐原则得出:

$$t_{opt} = \mu^Y - R_{opt}\mu^X \quad (18)$$

4.4.3 3D 配准-先验

在实际应用中, 数据可能有噪声或缺失, 单纯的 ICP 可能会算出错误的形状。引入正则项 E_{prior} 就是加入额外的约束条件使得 $E_{reg} = E_{match} + E_{prior}$ 。

全局刚性约束物体必须保持刚性变换, 不能形变。 $E_{prior} = \sum \|z_i - (Rx_i + t)\|^2$ 意味着我们寻找的对应点 z_i 必须能够通过一个严格的刚体变换 (R, t) 从原始点 x_i 得到。

线性模型是另一种更高级的先验, 允许物体在一定范围内发生形变, 但这种形变必须符合预定义的线性模型。在该线性模型 $s = Pd + m$ 中, m 为平均形状, P 为形变基, d 为需优化的形变参数。

4.5 网格表示与存储

网格由顶点集合 V 和面片集合 F 组成, 例如 $f_1 = (v_1, v_2, v_3)$ 。网格化 (Tessellation) 将多边形分割为三角形或四边形等基本图元, 三角化是最常见形式。

半边数据结构 (详见网格简化大作业)

HalfEdge: 指向顶点、对边、所属面与下一条半边 (按一定方向绕行得到)

Vertex: 坐标、法向、一条出边

Face: 一条所属半边

4.6 网格优化与平滑

4.6.1 孔洞填充

包括生成粗糙的三角化初始网格和通过优化与平滑提升网格质量两个步骤。

4.6.2 拉普拉斯平滑

基于物理扩散模型, 拉普拉斯平滑将网格顶点位置 p_i 的演化建模为扩散方程:

$$\frac{\partial p_i(t)}{\partial t} = \mu \Delta p_i(t)$$

其中 μ 为扩散常数², Δ 为离散拉普拉斯算子³。

采用显式欧拉法进行时间离散化, 得到迭代更新公式:

$$p_i^{(k+1)} = p_i^{(k)} + \lambda \Delta p_i^{(k)}$$

其中 $\lambda = \mu dt$ 为平滑因子, k 为迭代次数。

²扩散常数 μ : 控制平滑速率的参数, 与时间步长 dt 共同决定平滑强度 $\lambda = \mu dt$ 。

³拉普拉斯算子 Δ : 衡量顶点与邻域平均位置的差异, $\Delta p_i = \sum_{j \in \mathcal{N}(i)} w_{ij} (p_j - p_i)$ 。

5 数字几何处理一点云

5.1 点云的概念与特点

点云是最简单的三维几何表示，仅由一组 (x, y, z) 坐标构成，可能包含法向信息。这种表示方式无连接性，无法直接简化、细分或平滑渲染，也缺乏拓扑信息。但点云是多数三维扫描设备的直接输出格式，在三维重建、无人驾驶、场景感知等领域广泛应用。

5.2 点云获取技术

激光雷达通过发射短波激光脉冲并测量返回时间计算距离： $D = cT/2$ 。高精度时钟可实现约 0.15m 的测距精度，适用于远距离测量和地形扫描。

5.3 点云处理流程

典型流程包括形状扫描、多视角扫描对齐（配准）、平滑去噪、表面法向估计和表面重建五个步骤。

5.4 点一面 ICP 算法

经典的“点对点”模型假设源点云中的点在目标点云中有精确对应的点。然而在实际扫描中，点云往往是非均匀采样的。强行将点拉向最近的顶点会产生错误的切向力。改进的方法是让源点云中的点 x_i 贴合到目标点 y_i 所在的切平面上，允许点在平面内滑动。

给定目标点 y_i 处的单位法向量 n_{y_i} ，新的优化目标是 minimized 变换后的点到切平面的垂直距离：

$$E(R, t) = \sum_{i=1}^N ((Rx_i + t - y_i)^T n_{y_i})^2 \quad (19)$$

不同于点对点模型，点到面模型包含旋转矩阵与法向量的投影，无法直接通过 SVD 获得封闭解。通常采用线性化近似的方法求解，假设两次迭代之间的旋转角度很小。

假设旋转角度 $\alpha \approx 0$ ，则 $\cos \alpha \approx 1, \sin \alpha \approx \alpha$ 。旋转矩阵对向量的作用可以近似为（其中 r 为旋转向量）：

$$Rx_i \approx x_i + r \times x_i \quad (20)$$

将此近似代入损失函数：

$$E(r, t) \approx \sum_{i=1}^N ((x_i + r \times x_i + t - y_i)^T n_{y_i})^2 \quad (21)$$

为了求极值，令偏导数为 0，可以得到一个关于未知参数 $x = (r^T, t^T)^T$ （6 维向量）的线性方程组 $Ax = b$ 。

其中 A 是 6×6 矩阵， b 是 6×1 向量：

$$A = \sum_{i=1}^N \begin{pmatrix} x_i \times n_{y_i} \\ n_{y_i} \end{pmatrix} \begin{pmatrix} x_i \times n_{y_i} \\ n_{y_i} \end{pmatrix}^T \quad (22)$$

$$b = \sum_{i=1}^N ((y_i - x_i)^T n_{y_i}) \begin{pmatrix} x_i \times n_{y_i} \\ n_{y_i} \end{pmatrix} \quad (23)$$

通过求解该线性方程组，即可得到本次迭代的旋转向量 r 和平移向量 t 。

5.5 点云法向估计与去噪

5.5.1 数学原理

假设我们有一个点云表面样本。对于给定的查询点 P ，目标是找到该点处的切平面法向量 n ，使得点 P 的 k 个最近邻点 p_i 到通过 P 且以 n 为法向量的平面的距离平方和最小。

目标函数定义为最小化投影能量：

$$n_{opt} = \arg \min_{\|n\|=1} \sum_{i=1}^k ((p_i - P)^T n)^2 \quad (24)$$

为了求解上述约束优化问题（最小化目标函数，且满足约束 $n^T n = 1$ ），引入拉格朗日乘子 λ ：

$$\mathcal{L}(n, \lambda) = \sum_{i=1}^k ((p_i - P)^T n)^2 - \lambda(n^T n - 1) \quad (25)$$

对 n 求偏导并令其为 0：

$$\frac{\partial \mathcal{L}}{\partial n} = \sum_{i=1}^k 2(p_i - P)(p_i - P)^T n - 2\lambda n = 0 \quad (26)$$

整理后得到特征值方程：

$$Cn = \lambda n \quad (27)$$

其中矩阵 C 定义为：

$$C = \sum_{i=1}^k (p_i - P)(p_i - P)^T \quad (28)$$

结论：法向量 n 必须是矩阵 C 的特征向量。为了使目标函数值 $\sum ((p_i - P)^T n)^2 = n^T C n = n^T (\lambda n) = \lambda$ 最小，我们需要选择最小特征值对应的特征向量。

5.5.2 PCA 法向估计的具体步骤

在实际算法实现中（PCA 方法），也可以使用邻域的质心来代替查询点 P 以获得更稳定的协方差矩阵。具体步骤如下：

1. 对点云中的点 P ，设定半径 r 或邻居数量 k ，找到其最近邻点集 $\{p_1, \dots, p_k\}$ 。
2. 计算质心：

$$\bar{P} = \frac{1}{k} \sum_{i=1}^k p_i \quad (29)$$

3. 构建协方差矩阵：

$$C = \sum_{i=1}^k (p_i - \bar{P})(p_i - \bar{P})^T \quad (30)$$

4. 特征分解：对 3×3 的协方差矩阵 C 进行特征值分解（Eigen-decomposition）。

$$C = V\Lambda V^T, \quad \Lambda = \text{diag}(\lambda_1, \lambda_2, \lambda_3) \quad (31)$$

假设 $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq 0$ ，则 λ_3 对应的特征向量即为估计的法向量 n 。

5.5.3 参数选择与影响

- 关键参数 k/r ：由于采样会不均匀，故一般固定 r ，其大小直接影响估计质量。
- 曲率影响：如果 r 选得太大，会忽略细节（平滑过度），导致高曲率区域估计偏差。
- 噪声影响：如果 r 选得太小，会受到局部噪声的干扰，导致法向抖动。

5.6 点云去噪

去噪的目的：移除不靠近表面的点。利用局部邻域的协方差矩阵分析点云分布的几何特征，通过特征值判断该点是否为噪音。

5.6.1 基本原理

- 定义局部协方差矩阵 $C = \sum_{i=1}^k (p_i - P)(p_i - P)^T$ 。
- 对于任意单位向量 v ，二次型 $v^T C v = \sum ((p_i - P)^T v)^2$ 代表邻域点在 v 方向上的投影方差（能量）。
- 噪音特征：在法线方向上变化极小，最小特征值 $\lambda_{min} \approx 0$ 。点呈各向同性随机分布，特征值趋于相等，即 $\lambda_{max} \approx \lambda_{min}$ 。

5.6.2 算法流程

1. 对每个点 P 获取其 k 个最近邻点，计算协方差矩阵 C 并进行特征值分解，得到特征值 $\lambda_1 \geq \lambda_2 \geq \lambda_3$ 。
2. 计算特征值比率（如表面变化率 $\sigma = \frac{\lambda_3}{\lambda_1 + \lambda_2 + \lambda_3}$ ）或检查特征值分布。
3. 若局部特征值不满足表面几何特性（如 $\lambda_{max} \approx \lambda_{min}$ 或 λ_3 过大），则判定为噪音点并予以移除或平滑。

5.7 表面重建

5.7.1 隐式曲面方法

定义隐式函数 $f(x) = 0$ 表示曲面。对于点云，可构造符号距离函数 $f(x) = (x - p)^T n_p$ ，其中 p 为点云中距离 x 最近的点， n_p 为其法向量。需注意法向一致性对齐问题。

5.7.2 泊松表面重建

通过用点云法向场拟合指示函数 $\chi_M(p)$ 的梯度场 V 重建曲面，即 $\nabla \chi \approx V$ 。由于点云存在噪声和采样不均匀，直接由法向构建的向量场 V 通常不可积。通过应用散度算子到等式两边，转换为泊松方程 $\Delta \chi = \nabla \cdot V$ 。求解泊松方程得到平滑的隐式函数，再通过移动立方体算法提取等值面。

泊松表面重建不是局部地拼接表面，而是解一个全局方程，因此对噪声非常鲁棒（噪声被平滑掉了）。此外，泊松方程求解出的函数倾向于生成封闭的表面，非常适合 3D 打印等应用。

5.7.3 移动立方体算法

将空间划分为体素网格，根据网格顶点处的符号距离值确定表面穿越情况，通过预定义的 15 种拓扑连接方式生成三角网格。

5.8 点云深度学习：PointNet

点云的特性包括无序性（点集无固定顺序）、局部相关性（点与邻域点相互作用）和刚性变换不变性（旋转、平移不影响语义信息）。

PointNet 架构是首个直接处理无序点云的深度学习网络，能够用于点云数据的多种认知任务，如分类、语义分割和目标识别，具备置换不变性。核心思想是使用对称函数（如逐维度最大池化）聚合全局特征。网络结构包括共享权重的多层感知机、T-Net 对齐模块、最大池化层和全连接层。

6 网格光滑、简化、细分与检索

6.1 网格光滑（去噪）

6.1.1 去噪的目的与难点

网格光滑旨在滤除高频噪声，恢复几何表面的真实形状。这是一个病态逆问题：噪声类型与程度未知、真实几何特性未知，需依赖各种假设求解。常见假设包括曲面为 C^2 光滑、分片光滑或含大量平面、噪声为高斯噪声等。实际噪声复杂多样，这些假设常不成立，导致算法表现受限。

6.1.2 拉普拉斯光滑算法

一维曲线光滑的拉普拉斯算子定义为 $L(p_i) = \frac{1}{2}(p_{i-1} + p_{i+1}) - p_i$ 。迭代更新公式为 $p_i^{(t+1)} \leftarrow p_i^{(t)} + \lambda L(p_i^{(t)})$ ，其中 $0 < \lambda < 1$ 。推广到三维三角网格，对顶点 p_i 及其邻域 N_i ，拉普拉斯位移定义为 $\Delta p_i = \frac{1}{|N_i|} \sum_{j \in N_i} p_j - p_i$ ，迭代更新即可实现网格平滑。但需注意迭代次数过多可能导致网格收缩。

6.2 网格简化

网格简化是指用一些相对简单但维持几何特征的表示来近似一个给定的网格，包括用简单表示近似复杂网格以降低存储与传输开销、提高渲染与编辑效率、生成层次细节（LOD）模型适应不同视距绘制需求。

6.2.1 拓扑基本概念

- 亏格（genus）：曲面孔洞数目，如球体亏格为 0，环面亏格为 1
- 二维流形：局部拓扑等价于圆盘，三角网格中有且仅有两个三角形完全共边，每个三角形分别与三个相邻的三角形有且仅有一条共边。
- 带边界的二维流形：边界边仅属于一个三角形

6.2.2 简化方法分类

- 采样：直接对表面几何采样，适用于光滑表面。通常复杂且难于编程实现。
- 自适应细分：通过递归细分基底网格逼近原模型，在基底模型易于获取的情况下能取得最好效果。能够保持表面拓扑细节，因此有可能会降低对于大规模简化的处理能力。
- 去除：迭代删除顶点或面片，并对孔洞重新三角化。相对简单、易于编程实现并且运行效率高，限制局部变化且易于保持亏格，尤其精于处理移除几何冗余。

- 顶点合并：将多个顶点合并为一个，可改变拓扑结构。边坍塌算法易于保持局部拓扑，但是允许一般顶点合并操作的其他算法应能够修改拓扑并且聚合物体。

6.2.3 层次细节简化技术

层次细节简化技术：在不影响画面视觉效果的情况下，通过逐次简化景物的表面细节来减少场景的几何复杂性，提高绘制算法的效率。主要应用在简化采样密集的多面体网格激光扫描测距系统，扫描真实三维物体三维场景的存储、传输及绘制。

该技术对一个原始多面体模型建立几个不同逼近程度的几何模型。从近处观察物体时，采用精细的模型，从远处观察物体时，采用较粗糙的模型。当视点连续变化时，在相邻层次的模型之间形成光滑的视觉过渡，即几何形状过渡。因此，对层次细节简化技术研究集中在建立不同层次细节的模型，以及建立相邻层次的多边形网格之间的几何形状过渡。

6.2.4 基本化简操作

- 顶点删除：删除网格中的一个顶点，对它的相邻三角形形成的空洞作三角剖分
- 边压缩：网格上的一条边压缩为一个顶点，与该边相邻的两个三角形退化
- 面片收缩：网格上的一个面片收缩为一个顶点，该三角形本身和与其相邻的三个三角形都退化

6.2.5 网格模型简化方法

长方体滤波

首先建立多面体的长方体包围盒，并将该包围盒所包围的空间均匀剖分成一系列的小长方体子空间，然后采用各长方体子空间对景物顶点进行聚类合并，位于同一长方体空间的顶点被归于同一类。

缺点：顶点的合并导致了一些重要高频细节的丢失。

顶点删除的 **Schroeder** 局部判别准则

基于几何的“平坦性”来决定顶点的去留。对于网格内部的一个顶点 v ，其周围由一组邻接三角形面片集合 S 包围。为了判断 v 所在区域是否平坦，我们需要计算 v 到这一组面片构成的“平均平面”的距离。

设集合 S 中第 i 个三角形的面积为 A_i ，法向量为 $\mathbf{n}_i$ ，中心点为 $\mathbf{c}_i$ 。加权平均法向量 $\mathbf{N}$ 和加权平均中心 $\mathbf{C}$ 定义如下：

$$\mathbf{N} = \frac{\sum_{f \in S} \mathbf{n}_i A_i}{\sum_{f \in S} A_i}, \quad \mathbf{C} = \frac{\sum_{f \in S} \mathbf{c}_i A_i}{\sum_{f \in S} A_i} \quad (32)$$

顶点 v 的平坦度度量 d 为其到该平均平面的垂直距离：

$$d = |\mathbf{N} \cdot (v - \mathbf{C})| \quad (33)$$

若距离 d 小于用户设定的阈值 ϵ (即 $d < \epsilon$)，则认为该局部区域足够平坦，可以删除顶点 v ，并对留下的空洞进行三角剖分。

渐进网格简化

Hoppe 在渐进网格算法中，定义了一个显式的能量函数 $E(M)$ 来度量简化网格 M 与原始网格的逼近程度。网格优化的目标是通过迭代的边收缩操作 $(v_1, v_2) \rightarrow \bar{v}$ 及其逆变换（顶点分裂变换），最小化该能量函数。

能量函数由四部分组成，分别控制几何距离、网格正则度、标量属性和不连续特征：

$$E(M) = E_{dist}(M) + E_{spring}(M) + E_{scalar}(M) + E_{disc}(M) \quad (34)$$

$E_{dist}(M)$ 用于度量几何形状的逼近误差。定义为从原始网格上采样的稠密点集 $X = \{x_1, \dots, x_n\}$ 到当前简化网格 M 的距离平方和：

$$E_{dist}(M) = \sum_{i=1}^n d^2(x_i, \phi_V(|K|)) \quad (35)$$

其中， $\phi_V(|K|)$ 表示由当前顶点集 V 和拓扑结构 K 定义的连续网格表面， $d(\cdot)$ 表示点到表面的最短欧几里得距离。

$E_{spring}(M)$ 相当于在网格的每条边上放置一个弹性系数为 κ 的弹簧。该项用于正则化网格，防止顶点聚簇或长条形三角形的产生：

$$E_{spring}(M) = \sum_{\{j,k\} \in K} \kappa \|v_j - v_k\|^2 \quad (36)$$

其中 $\{j, k\} \in K$ 表示网格中的一条边， $\|v_j - v_k\|$ 为边的欧几里得长度。

$E_{scalar}(M)$ 度量网格标量属性（如颜色、纹理坐标、法向量）的拟合精度， $E_{disc}(M)$ ：度量视觉不连续特征（如明显的边界线、尖锐折痕、侧影轮廓线）的几何精度，确保简化后特征线不丢失。

二次误差度量

三维空间中的平面方程可表示为 $ax+by+cz+d=0$ ，满足归一化条件 $a^2+b^2+c^2=1$ 。定义平面参数向量 $\mathbf{p} = [a, b, c, d]^\top$ 。对于空间中任意一点 $\mathbf{v} = [x, y, z, 1]^\top$ ，其到平面 $\mathbf{p}$ 的距离平方 $D^2(\mathbf{v})$ 为：

$$D^2(\mathbf{v}) = (\mathbf{p}^\top \mathbf{v})^2 = \mathbf{v}^\top (\mathbf{p}\mathbf{p}^\top) \mathbf{v} = \mathbf{v}^\top \mathbf{K}_p \mathbf{v} \quad (37)$$

其中, $\mathbf{K}_p$ 是一个 4×4 的对称矩阵 (Fundamental Error Quadric):

$$\mathbf{K}_p = \mathbf{p}\mathbf{p}^\top = \begin{bmatrix} a^2 & ab & ac & ad \\ ab & b^2 & bc & bd \\ ac & bc & c^2 & cd \\ ad & bd & cd & d^2 \end{bmatrix} \quad (38)$$

对于网格上的顶点 v , 其总误差 $\Delta(v)$ 定义为它到所有关联平面集合 $planes(v)$ 的距离平方和:

$$\Delta(v) = \sum_{\mathbf{p} \in planes(v)} \mathbf{v}^\top \mathbf{K}_p \mathbf{v} = \mathbf{v}^\top \left(\sum_{\mathbf{p} \in planes(v)} \mathbf{K}_p \right) \mathbf{v} \quad (39)$$

定义顶点的二次误差矩阵 $\mathbf{Q}(v) = \sum \mathbf{K}_p$, 则误差函数简化为二次型:

$$\Delta(v) = \mathbf{v}^\top \mathbf{Q}(v) \mathbf{v} \quad (40)$$

当执行边收缩 $(v_1, v_2) \rightarrow \bar{v}$ 时, 新顶点 $\bar{v}$ 的误差矩阵遵循加法规则:

$$\bar{\mathbf{Q}} = \mathbf{Q}(v_1) + \mathbf{Q}(v_2) \quad (41)$$

我们需要寻找新位置 $\bar{\mathbf{v}} = [x, y, z, 1]^\top$, 使得误差 $\Delta(\bar{v}) = \bar{\mathbf{v}}^\top \bar{\mathbf{Q}} \bar{\mathbf{v}}$ 最小。这等价于求解偏导数为零的极值问题:

$$\frac{\partial \Delta}{\partial x} = 0, \quad \frac{\partial \Delta}{\partial y} = 0, \quad \frac{\partial \Delta}{\partial z} = 0 \quad (42)$$

结合齐次坐标约束 (第四分量为 1), 该问题转化为求解以下线性方程组:

$$\begin{bmatrix} q_{11} & q_{12} & q_{13} & q_{14} \\ q_{12} & q_{22} & q_{23} & q_{24} \\ q_{13} & q_{23} & q_{33} & q_{34} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad (43)$$

若系数矩阵可逆, 直接求解方程组得到最优位置 $\bar{v}$ 。若矩阵不可逆, 则在 v_1, v_2 和线段中点之间选择误差最小者作为新位置。

6.3 网格细分

细分的基本思想: 通过对控制网格进行迭代细化与平滑生成更光滑曲面。主要步骤包括细化阶段 (插入新顶点) 和平滑阶段 (调整顶点位置)。

6.3.1 Loop 细分

Loop 细分是首个三角网格细分方案，每一步将每个三角形三条边的中点连接，即细分为四个新三角形。细分规则：

- 原有顶点 p^k （度数为 n ）： $p^{k+1} = (1 - n\beta)p^k + \beta \sum_{i=0}^{n-1} p_i^k$
- 边中点插入新顶点 p_i^{k+1} ： $p_i^{k+1} = \frac{3p^k + 3p_i^k + p_{i-1}^k + p_{i+1}^k}{8}$

其中 $\beta(n) = \frac{1}{n} \left[\frac{5}{8} - \left(\frac{3}{8} + \frac{1}{4} \cos \frac{2\pi}{n} \right)^2 \right]$ 。

6.3.2 $\sqrt{3}$ 细分

Kobbelt 的 $\sqrt{3}$ 细分将每个三角形重心与顶点相连接，并将原三角形每条边翻转以连接相邻面心，每一步生成三个新三角形。更新规则：

- 面心顶点： $p_m^{k+1} = (p_a^k + p_b^k + p_c^k)/3$
- 原有顶点： $p^{k+1} = (1 - n\beta)p^k + \beta \sum_{i=0}^{n-1} p_i^k$

其中 $\beta(n) = \frac{4 - 2 \cos(2\pi/n)}{9n}$ 。

6.4 三维形状描述与检索

三维形状描述符用于在姿态变化下保持鲁棒性，主要分为全局与局部描述符。当前的方法要么是将 3D 图形的位姿对齐到标准坐标系，要么是使用以对象为中心的表示。常见方法包括 Spin Images、3D Shape Context、Light Field Descriptor 和 Shape Distributions。

6.4.1 Spin Images 与 3D Shape Context

Spin Images 使用圆柱坐标，在方位角方向池化，天然具有旋转不变性。3D Shape Context 使用球坐标，可通过球谐展开实现旋转不变性。两者均通过直方图统计局部几何分布。

6.4.2 Shape Distributions 与相似性度量

Shape Distributions 通过随机采样表面点对并计算其距离、角度等几何量的分布。相似性比较常用概率密度函数或累积分布函数的 L^n 范数、推土机距离或巴氏距离。

推土机距离（EMD）表达式为：

$$\text{EMD}(P, Q) = \min_F \sum_{i=1}^M \sum_{j=1}^N d_{ij} f_{ij}$$

其中 d_{ij} 为特征间距离， f_{ij} 为流动量。

7 网格变形与建模

7.1 网格变形概要

网格变形旨在通过对已有模型进行几何编辑，生成新模型或调整姿态，用于动画制作、形状建模等。核心挑战包括直观性（变形结果符合物理直觉）、高效性（支持实时交互）和灵活性（适用于不同表示）。

7.2 变形方法分类

曲面变形：模型定义为空壳，变形定义在模型表面上，与网格表示紧耦合，通常通过优化寻找某种“变形不变”的几何表示（如 Laplacian 坐标）。

空间变形：模型定义为体素，变形定义在模型邻域中，通过空间映射函数作用于模型。适用于任意几何表示，复杂度与模型表示无关。

7.3 基于 Laplacian 坐标的曲面变形

7.3.1 Laplacian 坐标定义

对于三角网格顶点 V ，Laplacian 坐标定义为 $\delta_i = L(v_i) = v_i - \frac{1}{d_i} \sum_{j \in N(i)} v_j$ ，其中 d_i 为顶点度数。Laplacian 坐标近似于顶点法向，具有平移不变性但无旋转不变性。

7.3.2 基本变形框架

给定控制顶点集合 C 及其目标位置 u_i ，变形通过求解最小二乘问题实现：

$$V' = \arg \min \sum_{i=1}^n \|\delta_i - L(v'_i)\|^2 + \sum_{i \in C} \|v'_i - u_i\|^2$$

该问题可转化为线性系统 $A^T A x = A^T b$ 求解⁴。

7.3.3 旋转不变性问题

Laplacian 坐标不具备旋转不变性，大旋转会导致形状扭曲。改进方法包括引入局部旋转变换 T_i ，使得 $T_i \delta_i \approx L(v'_i)$ ，并同时优化顶点位置 V' 与局部变换 T_i 。

⁴其伪逆解为

$$x = A^+ b = \begin{cases} (A^T A)^{-1} A^T b, & \text{若 } A \text{ 列满秩} \\ A^T (A A^T)^{-1} b, & \text{若 } A \text{ 行满秩} \end{cases}$$

采用 SVD 通用解法：

$$A = U \Sigma V^T \Rightarrow A^+ = V \Sigma^+ U^T$$
$$\Sigma^+ = \text{diag} \left(\frac{1}{\sigma_1}, \dots, \frac{1}{\sigma_r}, 0, \dots, 0 \right)$$

7.4 旋转传播方法

7.4.1 变形梯度与旋转提取

对于控制顶点处的仿射变换 $T(x) = Ax + t$ ，变形梯度 $\nabla T = A$ 可分解为旋转 R 与伸缩/错切 S ： $A = U\Sigma V^T \Rightarrow R = UV^T, S = V\Sigma V^T$ 。

7.4.2 光滑旋转传播

定义光滑权重场 $\alpha(x) \in [0, 1]$ ，通过指数映射与对数映射在流形上插值旋转：

$$R(x) = \exp(\alpha(x) \log(R_{\text{handle}}))$$

伸缩分量线性插值： $S(x) = \alpha(x)S_{\text{handle}}$ 。

7.5 空间变形：重心坐标法

7.5.1 基本思想

将模型置于多边形（2D）或多面体（3D）包围盒中，通过包围盒顶点的位移驱动内部点变形。变形函数表示为 $g(x) = \sum_{i=1}^n w_i(x)f_i$ ，其中 $w_i(x)$ 为重心坐标。

7.5.2 重心坐标的性质

包括单位分割 $\sum_i w_i(x) = 1$ 、线性精确性 $\sum_i w_i(x)x_i = x$ 和仿射不变性。

7.5.3 常见重心坐标

- 均值坐标：显式表达式，但仅在凸区域内非负
- 调和坐标：通过求解 Laplace 方程 $\nabla^2 w_i = 0$ 得到，处处非负且光滑，但无显式解

7.6 ARAP 变形（As-Rigid-As-Possible）

7.6.1 能量定义

ARAP 变形旨在保持局部区域的刚性，最小化每个顶点邻域的刚性变形误差：

$$E_{\text{ARAP}} = \sum_{i=1}^n \sum_{j \in N(i)} w_{ij} \|(v'_i - v'_j) - R_i(v_i - v_j)\|^2$$

其中 R_i 为局部旋转矩阵， w_{ij} 为余切权重。

7.6.2 迭代求解

可以从朴素拉普拉斯编辑作为最初猜想，通过交替优化求解：

1. 固定 v' ，求解 R_i ：对每个顶点邻域求解 Procrustes 问题⁵
2. 固定 R_i ，求解 v' ：转化为线性系统 $Lv' = b$ ，其中 L 为带权 Laplacian 矩阵

7.7 骨骼蒙皮动画

顶点位置由多个骨骼变换线性混合得到： $v'_i = \sum_{j=1}^m w_{ij} T_j v_i$ ，其中 w_{ij} 为蒙皮权重，满足 $\sum_j w_{ij} = 1$ 。权重可通过求解对应的有界双调和方程（Euler-Lagrange 方程 $\Delta^2 w_j = 0$ ）得到。

线性混合旋转会导致非刚性叠加，产生扭曲现象（糖果包装问题）。改进方法包括对偶四元数蒙皮等。

7.8 应用与前沿

包括数据驱动变形、无监督运动迁移、深度几何表示学习（如 SDM-NET）和动态场景编辑等方向。

RIMD (Rotation-Invariant Mesh Difference)

一种旋转不变的局部差分表示，将变形分解为旋转 R_i 与缩放 S_i 的序列，适用于大旋转变形。

ACAP (As-Consistent-As-Possible)

一种保持局部一致性的形变表示，通过优化旋转轴方向与角度一致性，支持高度非线性变形。

网格变分自编码器 (Mesh VAE)

将自编码器应用于网格表示，学习潜在空间中的形状分布。损失函数包括重建损失与 KL 散度。

局部形变分量分析

通过卷积自编码器学习稀疏局部形变基，损失函数包括重建损失、稀疏约束与非平凡项。

⁵Procrustes 问题（又称 Procrustes 分析或正交 Procrustes 问题）是寻找最优正交变换（旋转 + 反射）使两个点集对齐的数学问题。

NeRF 编辑 (NeRF-Editing)

将神经辐射场与显式网格编辑结合，步骤包括从 NeRF 提取显式网格、用户交互编辑、通过四面体化与 ARAP 将变形传播至辐射场、光线弯曲并渲染新视角。

高斯泼溅编辑 (SC-GS)

基于 3D Gaussian Splatting 的动态场景编辑方法，引入稀疏控制点与 ARAP 约束，支持实时运动编辑与高质量渲染。

8 光线跟踪

光线跟踪是一种基于光线传播物理过程的渲染算法，通过模拟光线从光源发出、经物体反射与折射后进入视点的路径，生成高度真实感的图像。其主要特点包括模拟真实光照（自然光、漫反射、镜面反射、折射）、真实材质（镜面、透明、半透明）和真实阴影（硬阴影、软阴影、透明阴影）。

8.1 光线跟踪的历史与发展

重要里程碑

- 1980 年：Turner Whitted 提出第一个完整的光线跟踪模型（Whitted 模型）
- 1984 年：Cook 等人提出分布式光线追踪（DRT），引入蒙特卡洛方法
- 1986 年：Kajiya 提出渲染方程与路径追踪算法

应用现状

离线渲染用于电影、动画制作；实时渲染随着 GPU 硬件加速（如 NVIDIA RTX）发展，逐步应用于游戏与交互式应用；混合渲染管线结合光栅化、计算着色器与光线追踪。

8.2 光线投射算法

光线投射是光线跟踪的前身，仅考虑首次相交，不支持阴影、反射、折射等效果。其对屏幕上每一像素：

1. 从视点通过像素中心发出一条光线，求其与场景中所有物体的交点
2. 按光线方向排序交点，选取最近交点
3. 使用局部光照模型计算该交点光亮度，并赋值给像素

当所有屏幕像素都处理完毕时，即得到一幅真实感图形。

8.3 光线跟踪算法原理

真实世界中光线从光源发出经多次弹射后进入人眼。光线跟踪采用逆向跟踪：从视点出发，通过像素发射光线，沿反射与折射方向递归追踪，直至光源或满足终止条件。

反射光线方向 $R = I - 2(I \cdot N)N$ 。折射方向根据 Snell 定律 $\eta_i \sin \theta_i = \eta_T \sin \theta_T$ 计算：

$$\cos \theta_T = \sqrt{1 - \frac{\eta_i^2(1 - \cos^2 \theta_i)}{\eta_T^2}}$$

折射方向：

$$T = -\frac{\eta_i}{\eta_T} I + \left(\frac{\eta_i}{\eta_T} (I \cdot N) - \cos \theta_T \right) N$$

该算法对每个像素：

1. 发射主光线，求与场景的最近交点
2. 计算局部光照（环境光、漫反射、镜面反射）
3. 若表面为反射面，发射反射光线并递归计算其颜色贡献
4. 若表面透明，发射折射光线并递归计算其颜色贡献
5. 叠加所有贡献作为像素颜色

终止条件：光线不与任何物体相交、贡献值低于阈值、递归深度达到上限。

8.4 光线与几何体求交

8.4.1 光线表示

光线采用参数形式： $P(t) = R_0 + tR_d (t > 0)$ ，其中 R_0 为起点， R_d 为光线的单位方向向量，参数 t 表示光线到达的位置。

8.4.2 光线与平面求交

平面方程为 $H(P) = n \cdot P + D = 0$ ，其中 n 为单位法向量。 P 到平面 H 的距离就是 $H(P)$ 。

将其与光线方程联立，解得 $t = -\frac{D+n \cdot R_0}{n \cdot R_d}$ ，需验证 $t > 0$ 。

8.4.3 光线与三角形求交

使用重心坐标法。设三角形顶点为 P_0, P_1, P_2 ，交点 P 可表示为 $P = (1 - \beta - \gamma)P_0 + \beta P_1 + \gamma P_2$ 。交点有效的条件为 $t > 0, 0 \leq \beta, \gamma \leq 1, \beta + \gamma \leq 1$ 。

联立得：

$$R_0 + tR_d = (1 - \beta - \gamma)P_0 + \beta P_1 + \gamma P_2$$

整理为：

$$(R_d \ P_0 - P_1 \ P_0 - P_2) \begin{pmatrix} t \\ \beta \\ \gamma \end{pmatrix} = P_0 - R_0$$

令：

$$E_1 = P_0 - P_1, \quad E_2 = P_0 - P_2, \quad S = P_0 - R_0$$

系统化为：

$$(R_d \ E_1 \ E_2) \begin{pmatrix} t \\ \beta \\ \gamma \end{pmatrix} = S$$

记 $D = \det(R_d, E_1, E_2)$ ，解得：

$$\begin{bmatrix} t \\ \beta \\ \gamma \end{bmatrix} = \frac{1}{D} \begin{bmatrix} \det(S, E_1, E_2) \\ \det(R_d, S, E_2) \\ \det(R_d, E_1, S) \end{bmatrix}$$

8.4.4 光线与球面求交

球面方程为 $\|P - P_c\| = r$ 。代入光线方程得二次方程

$$t^2 + 2t(R_d \cdot (R_0 - P_c)) + \|R_0 - P_c\|^2 - r^2 = 0$$

式中 R_d 为单位向量。记 $R_d \cdot (R_0 - P_c) = b$ ， $c = \|R_0 - P_c\|^2 - r^2$ ，则 $t = -b \pm \sqrt{b^2 - c}$ 。

几何法中，首先判断光线起点是否在球内部，随后找到光线上距离球心最近的点，判断这个最近距离与球半径关系即可。

8.4.5 光线与多边形求交

首先计算光线与多边形所在平面的交点，然后判断该交点是否在多边形内部（常用射线法，根据交点个数奇偶性判断）。

8.5 阴影计算

阴影通过阴影测试光线实现：从交点 P 向光源 L 发射一条光线，若该光线在到达光源前与场景中任意物体相交，则 P 处于阴影中。阴影因子 $f_i(P)$ 用于修正 Phong 光照模型中的漫反射光与镜面反射光项。

8.6 纹理映射

通过为表面顶点指定纹理坐标 (u, v) ，在光线与表面交点处通过插值获取纹理坐标，并从纹理图像中采样颜色。

8.7 光线跟踪加速技术

朴素光线跟踪需引出大量光线、遍历所有物体或面片，对每条光线与所有物体求交，时间复杂度为 $O(N \cdot \#rays)$ 。加速思路包括减少每条光线的求交测试次数和减少光线数量，采用分治思想将时间复杂度降为亚线性。

8.7.1 包围盒技术

为物体设置包围体（如轴对齐包围盒 AABB、可根据 PCA 构建的以特征向量为主轴的方向包围盒 OBB 等），光线先与包围盒求交，若不相交则跳过物体求交测试。合适的包围区域可以紧贴物体，更大程度地减少和物体不必要的相交测试。

8.7.2 层次包围盒（BVH）

将场景组织为二叉树结构。叶节点存储物体或三角面片；内部节点存储其子节点包围盒的并集。相交测试时对层次包围盒树执行搜索算法，从根节点开始递归测试，大幅减少无效测试。

层次包围盒的构建：先求得所有物体的包围盒，再递归地将其中物体分为两组，分别求出包围盒，直到递归深度或组中只有一个物体。

8.7.3 包围盒求交算法

基于 Slab（两个平行的平面）方法：长方体由三组 Slab 定义。对每个 Slab 计算光线进入与离开的参数 $t_{\min}^i$ 与 $t_{\max}^i$ ，整体相交条件为 $t_{\text{enter}} = \max_i t_{\min}^i, t_{\text{exit}} = \min_i t_{\max}^i, t_{\text{enter}} < t_{\text{exit}}$ 。即如果相交，则所有 $t_{\min}^i$ 全部小于 $t_{\max}^i$ 。

9 辐射度与渲染方程

9.1 预备知识

光照可分为局部光照（物体直接被光源照射）和全局光照（包括间接光照，如物体间的相互反射与透射）。

球面坐标 (r, θ, ϕ) 便于表示光线方向。立体角定义为 $d\omega = dA/r^2 = \sin\theta d\theta d\phi$ 。

面积微元 dA 在视线方向（与法向夹角 θ ）的可见面积为 $dA_{\text{proj}} = dA \cos\theta$ 。

辐射度量学基本量

- 光能 Q ：光子能量的总和，单位焦耳（J）
- 光通量 $\Phi = dQ/dt$ ：单位时间穿过截面的光能，单位瓦特（W）
- 辐照度 $E = d\Phi/dA$ ：单位面积接收的光通量，单位 W/m^2
- 辐出度 M ：单位面积发出的光通量
- 辐射率 L ：描述从微小面积沿微小方向辐射（或接收）的光通量

$$L = \frac{d\Phi}{dA \cos\theta d\omega} \quad [W/(m^2 \cdot sr)]$$

辐照度可表示为辐射率在入射半球上的积分： $E = \int_{\Omega} L(\omega) \cos\theta d\omega$ 。

9.2 双向反射分布函数（BRDF）

定义

BRDF 描述表面一点将入射光能从方向 ω_i 反射到方向 ω_r 的特性：

$$f_r(\omega_i \rightarrow \omega_r) = \frac{dL_r(\omega_r)}{dE_i(\omega_i)} = \frac{dL_r(\omega_r)}{L_i(\omega_i) \cos\theta_i d\omega_i}$$

单位： sr^{-1} 。

基本性质

- 可逆性（Helmholtz 可逆）： $f_r(\omega_i \rightarrow \omega_r) = f_r(\omega_r \rightarrow \omega_i)$
- 能量守恒：反射总能量不超过入射能量
- 线性性：BRDF 对入射光分布呈线性

9.3 BRDF 模型

经验模型

基于实验公式，计算简单但不一定满足物理定律。包括 Lambert 模型、Phong 模型和 Blinn-Phong 模型（用半角向量简化计算）。

基于物理的模型

建立在微平面理论基础上。Cook-Torrance 模型：

$$f_r(l, v) = \frac{F(l, h) G(l, v) D(h)}{4(n \cdot l)(n \cdot v)}$$

其中 F 为菲涅耳因子， D 为微平面法向分布函数（如高斯分布、Beckmann 分布）， G 为几何衰减因子， $h = (l + v) / \|l + v\|$ 为微表面法线。

扩展模型包括各向异性模型、Oren-Nayar 模型（粗糙漫反射）和波动光学模型（考虑衍射效应）。

数据驱动模型

通过测量大量真实材料 BRDF 数据，建立高维查找表或降维学习低维流形。

9.4 反射方程与渲染方程

9.4.1 反射方程

在表面点 x 处，出射辐射率 L_r 由自发光 L_e 与反射光组成：

$$L_r(x, \omega_r) = L_e(x, \omega_r) + \int_{\Omega} L_i(x, \omega_i) f_r(x, \omega_i, \omega_r) \cos \theta_i d\omega_i$$

9.4.2 渲染方程

考虑场景中所有表面间的相互反射：

$$L_r(x, \omega_r) = L_e(x, \omega_r) + \int_{\Omega} L_r(x', -\omega_i) f_r(x, \omega_i, \omega_r) \cos \theta_i d\omega_i$$

其中 $x' = \text{tr}(x, \omega_i)$ 为光线 (x, ω_i) 与场景的第一个交点。

9.4.3 算子形式与级数解

渲染方程可写为线性算子方程 $L = E + KL$ ，其级数解 $L = \sum_{k=0}^{\infty} K^k E$ 对应光线多次反射的累加。

9.5 蒙特卡洛路径追踪

9.5.1 蒙特卡洛积分

渲染方程无解析解，常用蒙特卡洛方法数值求解：

$$\int_{\Omega} f(\omega) d\omega \approx \frac{1}{N} \sum_{j=1}^N \frac{f(\omega_j)}{p(\omega_j)}$$

其中 ω_j 按概率密度 $p(\omega)$ 采样。

9.5.2 采样策略

包括均匀采样、BRDF 重要性采样（按 BRDF 分布采样反射方向）和光源重要性采样（直接对光源区域采样）。

9.5.3 路径追踪算法

递归估计辐射度，加入俄罗斯轮盘赌控制递归深度与方差。

9.5.4 拆分直接与间接光照

将渲染方程拆分为直接光照（0/1 次反射）与间接光照（多次反射）分别计算，提高效率。

9.6 全局光照效果与实现

通过路径追踪可自然实现软阴影、镜面反射与折射、漫反射间接光照（颜色渗出）和焦散等效果。

10 阴影与纹理

10.1 阴影

阴影是构建真实感图像的关键元素，提供物体位置与摆放的视觉提示。缺乏阴影会使场景难以理解。

10.1.1 阴影的定义

考虑一个由光源 L 照明的场景：

- 本影：从点 P 看不到光源的任何部分
- 半影：从点 P 能看到光源的一部分，但非全部
- 遮挡物：阻挡光线到达阴影区域的物体

10.1.2 阴影的类型

- 附着阴影：发生在接收者表面法向背离光源方向时
- 投射阴影：发生在接收者表面法向朝向光源，但光源被遮挡物遮挡时
- 自阴影：接收者与遮挡物属于同一物体

10.1.3 硬阴影与软阴影

硬阴影由点光源产生，表现为二值状态；软阴影由具有有限展度的光源（如面光源）产生，包含本影与半影区域，视觉效果更真实。

10.1.4 平面阴影

当阴影投射到平面上时，称为平面阴影。计算平面阴影的基本方法为阴影投影：使用矩阵将遮挡物上的点投影到目标平面。局限性：仅适用于平面接收者。

10.1.5 表面上的阴影

将阴影视为投影纹理：以光源为视点绘制场景，生成阴影图（黑白纹理），将该纹理投影到表面上，通过纹理坐标判断各点是否处于阴影中。缺点：物体无法将阴影投射到自身上。

10.1.6 阴影绘制算法

- 阴影域：由 Crow 提出，适用于任意表面。从点光源出发构建阴影体，通过射线与阴影体的交点数判断点是否处于阴影中。常用模板缓存实现计数。
- 阴影图：由 Williams 提出，基于深度缓存。以光源为视点生成深度图（阴影图），从相机视点绘制场景，比较片元深度与阴影图值。

10.2 纹理映射

纹理用于增强表面细节，避免复杂几何建模，提高渲染效率。

10.2.1 纹理采集

包括手工绘制、拍摄照片、过程纹理（程序生成）和纹理合成（从样本生成新纹理）。

10.2.2 纹理映射步骤

1. 纹理采集：获取或生成纹理图像
2. 纹理贴图：建立模型顶点与纹理坐标 (u, v) 的映射关系
3. 纹理滤波：防止走样，常用 Mipmapping 构建图像金字塔

10.2.3 映射方式

包括自然参数化（适用于规则形状）、手工指定和两步映射（先将纹理映射到简单中间表面，再映射到目标曲面）。

10.2.4 纹理滤波与反走样

走样因采样频率不足引起。Mipmapping 预计算多分辨率纹理金字塔，根据片元覆盖区域选择合适的层级进行采样，有效减少走样。

10.2.5 高级纹理技术

包括凹凸贴图（通过扰动法向量模拟表面细节）和高质量纹理建模（如 TM-Net，实现几何与纹理的联合生成）。

11 习题课：OpenGL 和光栅图形学实践

11.1 OpenGL 简介

OpenGL (Open Graphics Library) 是一个功能强大的开放图形库，前身是 SGI 公司开发的 IRIS GL。为了便于移植到不同硬件和操作系统，SGI 将其发展为开放标准。OpenGL 已成为跨平台的图形应用程序开发环境，广泛应用于 CAD/CAM、三维动画、数字图像处理及虚拟现实等领域。

11.1.1 发展历史与特点

- 起源：OpenGL 1.0 由 Mark Segal 和 Kurt Akeley 在 SGI 制定
- 重要版本：OpenGL 1.1 加入纹理对象；2.0 引入 GLSL；3.0+ 引入弃用机制，支持现代图形硬件
- 核心特点：跨平台性、网络透明性、高质量与高性能、出色的编程特性

11.1.2 OpenGL 的功能组成

包括模型绘制、变换、颜色与光照、纹理映射、位图与图像处理、动画、交互与反馈、特殊效果等功能。

11.2 OpenGL 体系结构与工作方式

11.2.1 体系结构层次

自底向上包括图形硬件、操作系统、窗口系统、OpenGL API 和应用软件。

11.2.2 流水线与状态机制

OpenGL API 调用被放入命令缓冲区，随后传递至流水线下一阶段。OpenGL 工作于状态机制，通过 `glEnable()/glDisable()` 设置状态。

11.2.3 OpenGL 组成与数据类型

包括核心库 `gl`、实用程序库 `glu`、编程辅助库 `aux`、实用工具包 `GLUT` 和 Windows 专用库 `wgl`。定义了 `GLbyte`, `GLshort`, `GLint`, `GLfloat`, `GLdouble` 等数据类型。

11.3 GLUT 工具包

GLUT 为 OpenGL 提供窗口管理、事件处理及高级几何物体创建功能。

11.3.1 初始化与窗口创建

包括 `glutInit()`, `glutInitWindowPosition()`, `glutInitWindowSize()`, `glutInitDisplayMode()`, `glutCreateWindow()` 等函数。

11.3.2 回调函数与事件循环

包括 `glutDisplayFunc()`, `glutMainLoop()`, `glutIdleFunc()`, `glutReshapeFunc()`, `glutKeyboardFunc()`, `glutMotionFunc()` 等。

11.3.3 菜单创建

GLUT 支持弹出式菜单,包括 `glutCreateMenu()`, `glutAddMenuEntry()`, `glutAddSubMenu()`, `glutAttachMenu()` 等函数。

11.4 OpenGL 绘图基础

11.4.1 清除窗口

`glClearColor()` 设置清除颜色, `glClear()` 清除指定缓冲区。

11.4.2 指定颜色与强制绘图

`glColor3f()` 设置当前绘图颜色, `glFlush()` 强制发送并执行 OpenGL 命令。

11.4.3 坐标系统与变换

包括视图变换、模型变换、投影变换和视口变换。矩阵操作包括 `glMatrixMode()`, `glLoadIdentity()`, `glTranslatef()`, `glRotatef()`, `glScalef()` 等。

投影包括透视投影 (`gluPerspective()`) 和正交投影 (`glOrtho()`)。视口设置使用 `glViewport()`。

11.5 光照处理

光照是增强三维场景真实感的关键。OpenGL 将光分为辐射光、环境光、漫反射光、镜面反射光, 允许设置多个光源。

11.5.1 光照模型与材质属性

光源属性包括位置、颜色、衰减系数等。材质属性包括环境光反射系数、漫反射光反射系数、镜面反射光反射系数、镜面指数和自发光颜色。

11.5.2 光照计算

在 RGBA 模式下，像素颜色由材料辐射、环境光贡献和各光源贡献（考虑衰减）计算得到。

11.6 几何图元绘制

11.6.1 顶点与图元定义

使用 `glVertex*()` 函数指定顶点，必须在 `glBegin()` 和 `glEnd()` 之间调用。图元类型由 `glBegin()` 的参数决定，包括点、线段、折线、三角形、四边形、多边形等。

11.6.2 绕法

定义多边形正面，默认逆时针为正面，可通过 `glFrontFace(GL_CW)` 改为顺时针。

11.6.3 基本图元绘制示例

包括点（`glPointSize()` 设置大小）、直线（使用 DDA 算法等实现）和多边形（必须为凸多边形，边不能相交）。

11.7 交互式绘图与菜单

通过 GLUT 实现鼠标交互绘图：注册鼠标点击回调记录起点、终点坐标；创建颜色与形状子菜单；根据存储的坐标、颜色、形状信息，在显示回调中批量绘制图形。

11.8 三维模型表达与读取

11.8.1 模型格式（OBJ）

OBJ 是一种文本格式的 3D 模型文件，包含顶点坐标（v）、纹理坐标（vt）、法向量（vn）和面定义（f）。

11.8.2 模型读取与渲染

可手动解析 OBJ 文件或使用 OpenMesh 库。渲染循环中将读取的数据传入 OpenGL，使用 `glBegin(GL_TRIANGLES)` 和 `glVertex3fv()`, `glTexCoord2fv()`, `glNormal3fv()` 绘制每个三角形面片。

11.9 综合示例：三维模型查看器

结合上述技术实现交互式三维模型查看器：初始化光照、材质、纹理、深度测试；从 OBJ 文件读取网格数据；通过键盘和鼠标控制模型显示；在显示回调中更新模型变换，重新绘制场景。

12 习题课：光线追踪加速

12.1 光线追踪加速的必要性

光线追踪计算复杂度高的原因包括光线数量庞大、物体数量众多和求交测试昂贵。朴素光线追踪算法时间复杂度为 $O(N)$ ，必须采用加速技术降低计算开销。

12.2 光线追踪加速的基本思路

加速的核心思路包括降低每条光线的追踪代价（减少相交测试次数）和减少需要追踪的光线数量（通过采样优化、重要性采样等方法）。

12.3 包围盒技术

包围盒技术用简单的几何体（如立方体、球体）完全包裹复杂物体，先测试光线与包围盒是否相交，若不相交则无需测试物体本身。

12.3.1 轴向包围盒（AABB）与方向包围盒（OBB）

AABB 的面与坐标轴平行，构建简单但包围紧密性较差；OBB 方向可随物体旋转，包围更紧密但求交测试稍复杂。

12.4 层次包围盒

层次包围盒（BVH）通过树形结构组织包围盒，将时间复杂度降低到亚线性（通常 $O(\log N)$ ）。

12.4.1 BVH 的数据结构与构建

BVH 是一棵二叉树：叶节点存储物体及其包围盒；内部节点存储子节点包围盒的并集。构建步骤包括计算物体包围盒、划分为两个子集、递归执行。

12.4.2 BVH 的相交测试

采用递归搜索：若光线与节点包围盒不相交则返回 false；若是叶节点则测试与所有物体的相交；否则递归测试左右子树。

12.5 空间分割技术

空间分割技术将场景空间划分为若干单元（体素），光线仅需遍历其穿过的单元。

12.5.1 均匀网格法

将场景包围盒均匀划分为 $N_x \times N_y \times N_z$ 个体素，每个体素记录与其相交的物体列表。使用 3D DDA 算法高效步进光线。优点：结构简单，构建快；缺点：对非均匀分布场景效率低下。

12.5.2 八叉树

均匀网格的自适应扩展，每个节点对应一个立方体区域，可递归 8 等分。优点：能对复杂区域更细分割；缺点：分割方式固定，可能产生大量空节点。

12.5.3 KD 树

灵活的空间二分树，每个内部节点沿某一坐标轴用分割平面将空间划分为两个子空间。节点结构包括分割轴、分割位置、左右子节点指针（内部节点）或物体列表、包围盒（叶节点）。优点：分割平面位置可调，能生成更平衡的树。

12.5.4 BSP 树

BSP 树是 KD 树的推广，分割平面可以是任意方向，在游戏工业中广泛应用，尤其适用于室内场景管理。

12.6 光线追踪实验代码分析

12.6.1 向量与几何类定义

包括 Vec_3 类（定义三维向量基本运算）和 Sphere 类（定义球体属性及相交测试函数）。

12.6.2 光线追踪递归函数

核心函数 trace 实现求交测试、法向量计算、递归计算反射与折射、直接光照与阴影、颜色合成等流程。

12.6.3 相机投影与渲染循环

使用针孔相机模型，根据像素位置计算光线方向。渲染循环对每个像素生成光线，调用 trace 函数计算颜色，输出为图像文件。

12.6.4 场景定义与结果

示例场景包含多个具有不同材质属性的球体及一个光源，可实现具有反射、折射及阴影效果的光线追踪图像。

12.7 总结

光线追踪加速是实现高效全局光照渲染的关键。包围盒与层次包围盒减少不必要的求交测试；空间分割技术利用空间连贯性优化遍历过程。需根据场景特点选择合适的加速结构，并结合现代硬件特性实现实时或交互式光线追踪。